

PYTHON

Unit 1: Introduction to Python

1. History of Python

Python is a high-level, interpreted, and general-purpose programming language that has become the backbone of modern software engineering.

- **The Developer:** Python was conceived and developed by **Guido van Rossum** at CWI (Centrum Wiskunde & Informatica) in the Netherlands.
- **Development Period:** The implementation of Python began in the late months of 1989.
- **Official Release:** The very first version, **Python 0.9.0**, was officially released on **February 20, 1991**.
- **Origin of the Name:** Contrary to popular belief, the language was not named after the snake. Guido van Rossum was a big fan of a popular BBC comedy series from the 1970s called **"Monty Python's Flying Circus."** He wanted a name that was short, unique, and slightly mysterious.
- **Major Milestone Versions:**
 - **Python 1.0:** Released in January 1994.
 - **Python 2.0:** Released in October 2000 (introduced features like list comprehensions and garbage collection).
 - **Python 3.0:** Released in December 2008. *Note: Python 3 was a major, backward-incompatible release designed to fix structural flaws in the language. Python 2 has since been officially deprecated.*

2. Features of Python

Python's massive popularity in the modern tech ecosystem is driven by its exceptional features:

- **Simple and Easy to Learn:** Python features a clean, highly readable syntax that closely resembles plain English. It eliminates complex syntactical constraints found in other languages, such as mandatory semicolons (;) or explicit curly braces {}, making it highly beginner-friendly.
- **Interpreted Language:** Python code is executed line-by-line by an interpreter. This eliminates the need for a separate compilation step before running, making development cycles faster and debugging far simpler.
- **Cross-Platform / Portable:** Python is machine-independent. A program written on a Windows machine can run seamlessly on Linux, macOS, or any other platform without modification.
- **Free and Open-Source:** Python is free to use, distribute, and modify. Its entire source code is publicly accessible to the global development community.
- **Dynamically Typed:** In languages like C, C++, or Java, variables must be explicitly declared with a data type before use. Python handles this dynamically; it determines the data type of a variable automatically at runtime based on the value assigned to it.
 - *Example:* Writing `x = 10` automatically classifies `x` as an integer.

- **Extensive Standard Library:** Python comes with a massive, pre-built library of modules and functions. This means developers do not need to write code from scratch for common tasks like mathematical computations, file I/O, or web interaction.
- **Supports Multiple Paradigms:** Python is highly versatile; it completely supports both **Procedure-Oriented Programming** (focused on functions) and **Object-Oriented Programming (OOP)** (focused on classes and objects).

3. Applications of Python

Python is utilized across almost every major sector of modern technology:

- **Web Development:** Powerful server-side frameworks like **Django** and **Flask** are used to build scalable, secure web applications. Major platforms like Instagram, YouTube, and Spotify rely heavily on Python.
- **Artificial Intelligence (AI) & Machine Learning (ML):** Python is the industry-standard language for AI/ML due to its specialized, high-performance libraries such as **TensorFlow**, **PyTorch**, and **Scikit-Learn**.
- **Data Science & Analytics:** Data scientists use libraries like **Pandas** (data manipulation), **NumPy** (numerical operations), and **Matplotlib / Seaborn** (data visualization) to clean and analyze complex datasets.
- **Automation & Scripting:** Python is ideal for automating repetitive tasks—such as batch file renaming, web scraping (extracting data from websites), or automated testing—using minimal code.
- **Desktop GUI Applications:** Tools like **Tkinter**, **PyQt**, and **Kivy** allow developers to build cross-platform desktop applications with robust graphical user interfaces.
- **Game Development:** Libraries like **Pygame** provide functional modules for rendering graphics, playing audio, and controlling physics, enabling the design of 2D games.

4. Python Installation Step-by-Step

To get Python running on a standard workstation, follow these steps:

1. Open your web browser and navigate to the official website: **www.python.org**.
2. Go to the **Downloads** section and select the appropriate package for your Operating System (Windows, macOS, or Linux). Download the latest stable release.
3. Once downloaded, run the executable setup installer (.exe file).
4. **CRITICAL STEP:** On the installation window, locate the checkbox at the bottom that reads **"Add python.exe to PATH"** and ensure it is checked/ticked. Failing to do this will prevent the Command Prompt from recognizing Python commands.
5. Click on **"Install Now"**. The wizard will copy files and configure your system.
6. **Verification:** To ensure Python was installed correctly, open your system's Command Prompt (CMD) or Terminal, type **python --version**, and press Enter. If installed properly, it will display the exact version number.

5. Python IDEs (Integrated Development Environments)

While you can write Python code in a basic text editor, standard development is typically done inside dedicated environments:

- **IDLE (Integrated Development and Learning Environment):** The default, lightweight environment that comes bundled with the standard Python installation. It is perfect for beginners executing their first code snippets.

- **VS Code (Visual Studio Code):** A powerful, highly customizable text editor created by Microsoft. When paired with the official Python extension, it becomes a world-class environment with advanced autocompletion and debugging features.
- **PyCharm:** A robust, dedicated Python IDE developed by JetBrains. It is packed with deep indexing, code analysis tools, and built-in testing environments, making it the preferred choice for large-scale enterprise development.
- **Jupyter Notebook:** A web-based interactive computational environment. It allows developers to mix live code, equations, visualizations, and narrative text in a single document, making it highly popular in Data Science.

6. Python Execution Modes

Python programs can be interacted with and executed using two primary modes:

A. Interactive Mode

- In this mode, developers type code directly into the Python Shell shell, and the interpreter executes each line immediately upon pressing Enter.
- **Primary Use:** Ideal for rapid prototyping, quick math calculations, or testing how specific lines of code behave.
- **Accessing Interactive Mode:** Open CMD/Terminal, type `python`, and press Enter. The prompt will switch to three greater-than signs (`>>>`), indicating it is ready for your statements.
- *Limitation:* Code written in this mode is stored strictly in volatile memory. Once the window is closed, the code is permanently lost.

B. Script Mode

- In this mode, developers write their entire program inside a file and save it with a `.py` extension.
- **Primary Use:** This is the standard method used for building real-world applications, reusable software tools, and complex scripts.
- **Accessing Script Mode:** Open a code editor (like VS Code or Notepad), type out your logic, save it as `program.py`, and run the entire file at once through the terminal or your IDE execution controls.

7. First Python Program

Writing a "Hello World" or welcome script in Python requires just a single line of text, showcasing its straightforward syntax.

The Code:

```
# This is a single-line comment outlining our first program  
print("Welcome to Open Technology Computer Education!")
```

Syntax Breakdown:

1. **The Hash Symbol (#):** Tells the Python interpreter that the text following it on that line is a comment. The interpreter skips this line completely during execution.
2. **print() Function:** A built-in Python function used to transmit data streams out to the standard output device (the screen).
3. **Double Quotes ("..."):** Encloses a sequence of characters, creating a data structure known as a **String**. Single quotes ('...') can also be used interchangeably.

Unit 2: Python Fundamentals

Python Fundamentals comprise the core syntax rules, lexical terms, and foundational architectures required to construct valid programs.

1. Keywords in Python

Keywords are system-reserved words that carry a predefined, immutable structural meaning for the Python compiler and interpreter.

- **Essential Rules for Keywords:**
 - Keywords **cannot** be reused as names for variables, functions, identifiers, or any other user-defined objects.
 - Python is strictly case-sensitive. Keywords must match their specific case mapping exactly.
 - Modern Python releases contain approximately **35 structural keywords**.
 - *Important Note:* The vast majority of keywords are entirely lowercase. Only three keywords begin with an uppercase letter: `True`, `False`, and `None`.
- **Examples of Essential Keywords:**
 - **Control Flow:** `if`, `else`, `elif`
 - **Loops:** `for`, `while`, `break`, `continue`
 - **Structural Components:** `def`, `return`, `class`
 - **Namespace Management:** `import`, `from`, `as`

2. Identifiers in Python

An identifier is a user-defined token name assigned to programmatic elements such as variables, functions, classes, arrays, or custom modules.

- **Strict Structural Rules for Designing Identifiers:**
 1. Identifiers can consist of any combination of uppercase letters (A-Z), lowercase letters (a-z), digits (0-9), and underscores (`_`).
 2. The initial character of an identifier **must** be a letter or an underscore (`_`). It **cannot** begin with a number.
 - *Valid:* `student_age`, `data1`, `_temp_val`
 - *Invalid:* `1student`, `5data`
 3. Identifiers cannot contain special characters or symbols, such as `@`, `$`, `%`, `!`, or white spaces.
 - *Invalid:* `user$id`, `total score`
 4. Identifiers cannot match any system-reserved keywords.

3. Variables in Python

A variable is a named storage location in the computer's memory (RAM) used to hold data values that can be manipulated or changed during program execution.

- **Understanding Dynamic Typing:**
- Unlike statically typed languages where you must declare a variable's data type before assigning data (e.g., `int x = 5;`), Python uses **Dynamic Typing**. The Python interpreter determines the appropriate data type automatically based on the object assigned to the variable.
- **Variable Initialization and Multi-Assignment:**
- The single equals sign (`=`) acts as the Assignment Operator in Python.

```
# Variable Assignment and Initialization

age = 22          # Automatically inferred as an Integer (int)

course_fee = 4500.50 # Automatically inferred as a Floating-Point (float)

center_name = "Open Tech" # Automatically inferred as a String (str)


# Multi-Variable Assignment

x = y = z = 100    # Assigns a single value to multiple variables simultaneously

a, b, c = 5, 10, 15 # Assigns distinct values to multiple variables in a single line
```

4. Constants in Python

A constant is an identifier containing a value that remains fixed and cannot be altered or overwritten throughout the duration of a program's lifecycle.

- **How Python Handles Constants:**
- Python does not possess built-in architectural constraints or keywords (like `const` or `final`) to lock a variable from modification.
- **The Developer Convention:**
- To define a constant, Python developers follow a strict naming convention: the identifier is written entirely in **UPPERCASE LETTERS**. This informs other programmers that the variable's value should remain unchanged.

```
# Constants are defined using uppercase names

PI = 3.14159

GRAVITY = 9.81

MAX_ENROLLMENTS = 250
```

Note: If code later reassigns a value to `PI`, the program will execute without error, but doing so violates standard programming best practices.

5. Comments in Python

Comments are explanatory notes embedded directly inside the source code to make it human-readable. They are entirely ignored by the Python interpreter during runtime.

- **Types of Comments:**
 - **A. Single-Line Comments:** Indicated by placing a hash symbol (`#`) at the start of the comment. Everything to the right of the symbol on that line is treated as a comment.

```
# This entire line is a comment explaining the print statement

print("Python Fundamentals") # Inline comment explaining this statement
```

○

- **B. Multi-Line Comments / Docstrings:** While Python does not have an explicit multi-line comment symbol, developers can comment out large blocks of text by adding a `#` to each line, or by enclosing the text within **Triple Quotes** (`"""` or `'''`).

```
"""  
  
Open Technology Computer Education  
  
Python Core Curriculum Notes  
  
Unit 2: Comprehensive Fundamentals Reference Sheet  
  
"""  
  
print("Standard execution block initialized.")
```

○

6. Input and Output Statements

Data processing applications require mechanisms to ingest data from an input device (Input) and display the processed results to the user (Output).

A. Output Statement: The `print()` Function

The built-in `print()` function formats data objects into text streams and writes them out to the display screen.

- **Standard Output Variations:**

```
# 1. Outputting a Simple String Literal  
  
print("Open Technology Computer Education")  
  
# 2. Outputting a Pure Variable State  
  
student_count = 60  
  
print(student_count)  
  
# 3. Concatenating Text and Variables Using Comma Separators  
  
print("Active Student Roster Count:", student_count)  
  
# 4. Utilizing Modern F-Strings (Formatted String Literals)  
  
print(f"There are currently {student_count} active students registered.")
```

B. Input Statement: The `input()` Function

The built-in `input()` function pauses program execution and waits for the user to type text on the keyboard and press Enter.

- **THE CORE RULE FOR INPUT PROCESSING:**
- The `input()` function captures user inputs and reads them strictly as a **String (text data structure)**. Even if a user enters numeric inputs (such as `45`), Python stores that value as text ("`45`"), making mathematical calculations impossible without conversion.
- **Typecasting Inputs:**
- To perform calculations on input values, developers must use conversion functions like `int()` or `float()` to convert strings into numeric data types.

```
# 1. Ingesting Standard String Input (No conversion required)

student_name = input("Enter student first name: ")

print(f"Record created for student: {student_name}")


# 2. Ingesting Numeric Inputs Using Typecasting

# Wrapping int() around input() explicitly converts the text into a number

first_exam_score = int(input("Enter score for Exam 1: "))

second_exam_score = int(input("Enter score for Exam 2: "))


# Mathematical evaluation is now fully enabled

composite_total = first_exam_score + second_exam_score

print(f"Calculated Composite Assessment Total: {composite_total}")
```

Here are the complete, modern, and highly structured study notes for **Unit 3: Data Types** in clean English. The formatting has been kept clean, consistent, and well-organized so you can copy and paste them directly into Microsoft Word or Excel for your students.

Unit 3: Data Types

In programming, a **Data Type** specifies the type of value a variable can hold and what kind of mathematical, relational, or logical operations can be applied to it without causing an error.

1. Numeric Data Types

Python provides three built-in numeric data types to represent numbers in various formats: **Integer**, **Float**, and **Complex**.

A. Integer (`int`)

- Integers represent whole numbers without any fractional or decimal parts.
- They can be positive, negative, or zero.

- unlike many other languages (like C or Java) which have limit boundaries for numbers, Python's `int` has arbitrary precision—meaning it can store arbitrarily large numbers as long as your computer has enough memory.
 - Example: `x = 500, y = -25, z = 0`

B. Floating-Point (`float`)

- Floats represent real numbers that contain a decimal point or fractional component.
- They can also be written in scientific notation using the letter `e` or `E` to represent powers of 10.
 - Example: `price = 99.95, pi = 3.14159, scientific_val = 2.5e3` (which equals $2.5 \times 10^3 = 2500.0$)

C. Complex Numbers (`complex`)

- Complex numbers are written in the form `$a + bj$`, where `$a$` is the **real part** and `$b$` is the **imaginary part**.
- In Python, the lowercase or uppercase letter `j` is strictly used to denote the imaginary unit (instead of the traditional mathematical symbol `i`).
 - Example: `num = 3 + 5j`

2. String Data Type (`str`)

A string is an ordered sequence of characters wrapped inside quotation marks. It is used to store text-based data.

- **Declaration Rules:**
 - Strings can be enclosed using single quotes (`'...'`), double quotes (`"..."`), or triple quotes (`"""..."""` or `'''...'''`).
 - Triple quotes are specifically used to define **multi-line strings** or docstrings.
- **Key Characteristics:**
 - **Immutability:** Python strings are *immutable*, meaning that once a string is created, its individual characters cannot be changed or modified in place.
 - **Indexing:** Each character in a string has an index starting from `0` for the first character (left-to-right). Python also supports negative indexing starting from `-1` for the last character (right-to-left).

```
# String Examples

message = "Welcome to Open Technology"

multi_line = """This string spans
across multiple lines
in Python."""

# Accessing characters via indexing

word = "Python"

print(word[0]) # Output: 'P'
```

```
print(word[-1]) # Output: 'n' (Last character)
```

3. Boolean Data Type (`bool`)

The Boolean data type represents one of two built-in logical truth values: `True` or `False`.

- **Key Characteristics:**

- Booleans are essential for conditional expressions, branching logic (`if-else`), and loop structures.
- *Case Sensitivity:* The first letter must always be capitalized (`True` and `False`). Writing `true` or `false` in lowercase will result in a `NameError`.
- Internally, Python treats `True` as having a numeric value of `1` and `False` as having a numeric value of `0`.

```
# Boolean Expressions

is_logged_in = True

has_paid_fees = False

# Evaluated Output

print(10 > 5) # Output: True

print(3 == 4) # Output: False
```

4. Type Conversion (Implicit Type Conversion)

Type Conversion—also known as **Implicit Type Conversion** or **Coercion**—occurs when the Python interpreter automatically changes one data type to another without any manual intervention from the programmer.

- **How it works:**

- Python performs implicit conversion during operations involving mixed data types to prevent data loss.
- For instance, if you add an integer to a floating-point number, Python automatically upgrades the integer to a float because a float can hold more precise decimal data.

```
# Implicit Type Conversion Example

num_int = 15 # Integer type

num_float = 4.5 # Float type

# Python automatically converts num_int to a float before performing addition

result = num_int + num_float
```

```
print(result)    # Output: 19.5  
print(type(result)) # Output: <class 'float'>
```

5. Type Casting (Explicit Type Conversion)

Type Casting—also known as **Explicit Type Conversion**—is the manual process where the programmer forces a variable to change from its current data type into a completely different one.

- **How it works:**
 - This is achieved by wrapping the variable or value inside built-in Python type constructor functions.
- **Common Type Casting Functions:**
 - `int()` - Converts a compatible float or string into an integer (truncates decimal values).
 - `float()` - Converts an integer or compatible string into a floating-point decimal.
 - `str()` - Converts any data type (numbers, booleans, etc.) into its equivalent string textual representation

```
# Explicit Type Casting Examples  
  
# 1. Float to Integer (Decimals are removed)  
gpa = 3.95  
rounded_gpa = int(gpa)  
print(rounded_gpa) # Output: 3  
  
# 2. String to Integer (Crucial for user inputs)  
age_string = "25"  
age_numeric = int(age_string)  
print(age_numeric + 5) # Output: 30  
  
# 3. Number to String (For clean text display)  
roll_no = 101  
display_text = "Student Roll Number is " + str(roll_no)  
print(display_text) # Output: Student Roll Number is 101
```

Unit 4: Operators

In programming, an Operator is a special symbol that tells the compiler or interpreter to perform specific mathematical, relational, or logical manipulations on data values. The data values that operators work upon are called Operands.

- *Example:* In the expression $x + y$, $+$ is the Operator and x and y are the Operands.

1. Arithmetic Operators

Arithmetic operators are used to perform common mathematical calculations such as addition, subtraction, multiplication, and division.

Operator	Name	Description	Example (a=10,b=3)	Result
+	Addition	Adds two operands together.	$a + b$	13
-	Subtraction	Subtracts the right operand from the left operand.	$a - b$	7
*	Multiplication	Multiplies two operands.	$a * b$	30
/	Division	Divides the left operand by the right operand; always returns a float.	a / b	3.33333333

<code>%</code>	Modulus	Divides operands and returns the remainder.	<code>a % b</code>	<code>1</code>
<code>**</code>	Exponentiation	Performs "power" calculation (left operand raised to the power of right).	<code>a ** b</code>	<code>1000</code> (10^3)
<code>//</code>	Floor Division	Divides operands and returns the lowest whole number (removes decimals).	<code>a // b</code>	<code>3</code>

2. Relational Operators (Comparison Operators)

Relational operators are used to compare two values. The result of a relational evaluation is always a Boolean value—either `True` or `False`.

Operator	Name	Description	Example (a=10,b=3)	Result
<code>==</code>	Equal to	Returns <code>True</code> if both operands are equal.	<code>a == b</code>	<code>False</code>

<code>!=</code>	Not Equal to	Returns <code>True</code> if operands are not equal.	<code>a != b</code>	<code>True</code>
<code>></code>	Greater than	Returns <code>True</code> if the left operand is larger than the right.	<code>a > b</code>	<code>True</code>
<code><</code>	Less than	Returns <code>True</code> if the left operand is smaller than the right.	<code>a < b</code>	<code>False</code>
<code>>=</code>	Greater than or equal to	Returns <code>True</code> if the left operand is greater than or equal to the right.	<code>a >= 10</code>	<code>True</code>
<code><=</code>	Less than or equal to	Returns <code>True</code> if the left operand is less than or equal to the right.	<code>a <= b</code>	<code>False</code>

3. Logical Operators

Logical operators are used to combine multiple conditional or relational expressions together. They evaluate statements based on Boolean logic.

- **and** (Logical AND): Returns **True** only if both conditions are true. If even one condition is false, it returns **False**.
- **or** (Logical OR): Returns **True** if at least one of the conditions is true. It returns **False** only if all conditions are false.
- **not** (Logical NOT): Reverses the logical state of its operand. It turns **True** into **False**, and **False** into **True**.

```
x = 5
y = 10

print(x > 2 and y < 15) # True and True -> Output: True
print(x > 10 or y < 15) # False or True -> Output: True
print(not(x == 5))      # not(True)    -> Output: False
```

4. Assignment Operators

Assignment operators are used to assign values from the right side of the expression to the variable on the left side. Python supports compound assignment shorthand operations known as Shortcut/Augmented Assignment Operators.

Operator	Example	Equivalent To
=	<code>x = 5</code>	<code>x = 5</code>
+=	<code>x += 3</code>	<code>x = x + 3</code>
-=	<code>x -= 2</code>	<code>x = x - 2</code>

<code>*=</code>	<code>x *= 4</code>	<code>x = x * 4</code>
<code>/=</code>	<code>x /= 2</code>	<code>x = x / 2</code>
<code>%=</code>	<code>x %= 3</code>	<code>x = x % 3</code>
<code>//=</code>	<code>x //= 2</code>	<code>x = x // 2</code>
<code>**=</code>	<code>x **= 2</code>	<code>x = x ** 2</code>

5. Bitwise Operators

Bitwise operators perform mathematical calculations directly on the binary patterns (bits) of integers. They convert decimal values to binary equivalents, perform the bit-by-bit calculation, and return a decimal answer.

- Let \$a = 6\$ (Binary: `0110`) and \$b = 3\$ (Binary: `0011`).

Operator	Name	Description	Example Result

<code>&</code>	Bitwise AND	Sets each bit to <code>1</code> if both bits are <code>1</code> .	<code>a & b</code> $\rightarrow$ <code>2 (0010)</code>
<code> </code>	Bitwise OR	Sets each bit to <code>1</code> if at least one bit is <code>1</code> .	<code>a b</code> $\rightarrow$ <code>7 (0111)</code>
<code>^</code>	Bitwise XOR	Sets each bit to <code>1</code> if only one of the bits is <code>1</code> .	<code>a ^ b</code> $\rightarrow$ <code>5 (0101)</code>
<code>~</code>	Bitwise NOT	Inverts all the bits (flips <code>1</code> to <code>0</code> and vice versa). Evaluates to $-(n+1)$.	<code>~a</code> $\rightarrow$ <code>-7</code>
<code><<</code>	Zero-fill Left Shift	Shifts the binary sequence to the left by pushing zeros in from the right.	<code>a << 1</code> $\rightarrow$ <code>12 (1100)</code>

>>	Signed Right Shift	Shifts the binary sequence to the right by pushing copies of the leftmost bit.	$a \gg 1$ $\rightarrow$ 3 (0011)
----	--------------------	--	--

6. Membership Operators

Membership operators are unique to languages like Python. They are used to test whether a value or variable is found inside a sequence collection (such as a string, list, tuple, or dictionary).

- **in:** Returns **True** if the specified value is present inside the given sequence collection.
- **not in:** Returns **True** if the specified value is absent/missing from the given sequence collection.

```
course_title = "Python Programming"

scores = [10, 20, 30, 40]

print("Python" in course_title) # Output: True
print("Java" in course_title)  # Output: False
print(50 not in scores)       # Output: True
```

7. Identity Operators

Identity operators compare the memory locations of two objects. They do not check if the values are equal; instead, they check if both variables point to the exact same object block in the computer's memory (RAM).

- **is:** Returns **True** if both variables point to the exact same object memory address.
- **is not:** Returns **True** if both variables point to completely separate object memory addresses.

Key Difference: **==** compares whether values are identical, while **is** compares whether memory tracking IDs (**id()**) are identical.

```
list1 = [1, 2, 3]

list2 = [1, 2, 3]

list3 = list1 # Pointing to the same memory space
```

```
print(list1 == list2) # True (Their data contents are identical)

print(list1 is list2) # False (They are saved in different memory boxes)

print(list1 is list3) # True (They reference the exact same memory box)
```

Unit 5: Decision Making Statements

In programming, Decision Making Statements (also known as Conditional Statements or Control Flow Statements) are used to control the order of execution of a program based on certain conditions.

By default, code executes sequentially (line-by-line from top to bottom). Decision-making statements allow a program to evaluate a condition as either **True** or **False** and choose different execution paths depending on the outcome.

1. The **if** Statement

The **if** statement is the simplest form of decision-making. It is used to execute a block of code only if a specified condition evaluates to **True**. If the condition is **False**, the block of code inside the **if** statement is completely skipped.

- **Understanding Indentation:** Unlike languages like C, C++, or Java that use curly braces **{ }** to define blocks of code, Python relies strictly on indentation (whitespace or a tab at the beginning of a line). All lines indented by the same number of spaces form a single block of code.

Syntax:

```
if condition:

    # Code to execute if the condition is True

    statement1

    statement2
```

Practical Example:

```
# Program to check if a student passed

marks = 45

if marks >= 33:

    print("Result: Passed")

    print("Congratulations!")
```

```
print("Program finished.")
```

```
# The last line prints regardless because it is outside the indented if-block.
```

2. The if-else Statement

The `if-else` statement provides two distinct paths of execution. If the given condition is `True`, the code block under the `if` statement runs. If the condition evaluates to `False`, the code block under the `else` statement runs instead.

Syntax:

```
if condition:
    # Executes if condition is True
    statement_block_1
else:
    # Executes if condition is False
    statement_block_2
```

Practical Example:

```
# Program to check if a number is Even or Odd

number = int(input("Enter an integer: "))

if number % 2 == 0:
    print(f"{number} is an Even number.")
else:
    print(f"{number} is an Odd number.")
```

3. Nested if Statements

A Nested if statement refers to placing an `if` or `if-else` block inside another `if` or `if-else` block. This architecture is utilized when multiple sequential layers of conditions must evaluate to `True` before an action can be performed.

Syntax:

```
if condition1:
    # Executes if condition1 is True
    if condition2:
```

```
# Executes only if BOTH condition1 and condition2 are True

statement_block_A

else:

    # Executes if condition1 is True but condition2 is False

    statement_block_B

else:

    # Executes if condition1 is False

    statement_block_C
```

Practical Example:

```
# Program to check user authentication and access level

username = input("Enter username: ")
password = input("Enter password: ")

if username == "admin":

    if password == "secure123":

        print("Login Successful! Access granted to Admin Dashboard.")

    else:

        print("Login Failed: Incorrect Password.")

else:

    print("Login Failed: User not found.")
```

4. The `if-elif-else` Ladder

The `if-elif-else` ladder is used when a program needs to check multiple distinct conditions one after another. `elif` is short for "else if".

- **Execution Behavior:** Python evaluates conditions from top to bottom. As soon as it finds a condition that is `True`, it executes that specific block of code and skips the rest of the ladder entirely. If absolutely none of the conditions are `True`, the catch-all `else` block at the very bottom executes.

Syntax:

```
if condition1:
```

```
statement_block_1

elif condition2:

    statement_block_2

elif condition3:

    statement_block_3

else:

    # Runs if all above conditions fail

    fallback_statement_block
```

Practical Example:

```
# Student Grading System Program

score = int(input("Enter student exam marks (0-100): "))

if score >= 90:

    grade = "A+"

elif score >= 80:

    grade = "A"

elif score >= 70:

    grade = "B"

elif score >= 60:

    grade = "C"

elif score >= 33:

    grade = "D (Passed)"

else:

    grade = "F (Failed)"

print(f"Final Assigned Grade: {grade}")
```

Unit 6: Looping Statements

In programming, Looping Statements (also known as Iteration Statements or Repetitive Control Structures) are used to execute a specific block of code multiple times continuously as long as a specified condition remains **True**.

Loops eliminate the need to write the exact same code repeatedly, drastically reducing code redundancy, saving memory space, and making programs much more efficient.

1. The **while** Loop

The **if** statement checks a condition once, but a **while** loop checks the condition repeatedly. It is an entry-controlled loop, meaning the condition is evaluated *before* entering the loop body. If the initial condition is **False**, the loop body will not execute even once.

- **Key Requirement:** A **while** loop must always have a mechanism (like an increment or decrement statement) inside its body to eventually make the condition **False**. If the condition never becomes **False**, it creates an infinite loop, which can freeze or crash the program.

Syntax:

```
while condition:
    # Body of the loop
    statements
    iteration_update (increment/decrement)
```

Practical Example:

```
# Program to print numbers from 1 to 5
counter = 1 # 1. Initialization

while counter <= 5: # 2. Condition Check
    print(f"Iteration Number: {counter}")
    counter += 1 # 3. Increment (Update)

print("Loop completed successfully.")
```

2. The **for** Loop

Unlike other programming languages where a **for** loop functions just like a counter-based loop, Python's **for** loop is designed to iterate over the items of any sequence (such as a list, a tuple, a string) or through a range of numbers using the built-in **range()** function.

Understanding the **range() Function:**

The **range()** function generates a sequence of integers dynamically. It can take up to three parameters: **range(start, stop, step)**

- **start:** The starting value of the sequence (Default is **0** if omitted).
- **stop:** The ending limit of the sequence. Note: This upper limit is exclusive and not included in the output.
- **step:** The value by which the counter increases each time (Default is **1** if omitted).

Syntax:

```
for variable in sequence:
```

```
# Body of the loop
statements
```

Practical Examples:

```
# Example A: Iterating through a fixed range of numbers (0 to 4)
```

```
for i in range(5):
    print("Value of i:", i)
```

```
# Example B: Using custom start, stop, and step arguments (Print odd numbers 1 to 9)
```

```
for num in range(1, 10, 2):
    print("Odd Number:", num)
```

```
# Example C: Iterating through characters of a String
```

```
center_name = "Open"
for char in center_name:
    print("Character:", char)
```

3. Nested Loops

A Nested Loop refers to a structural design where one loop statement is placed completely inside the body of another loop statement. You can place a `for` loop inside a `while` loop, or vice versa, or nest `for` loops together.

- **Execution Rule:** For every single iteration of the outer loop, the inner loop executes completely through its entire cycle from start to finish.

Syntax:

```
for outer_variable in outer_sequence:
    for inner_variable in inner_sequence:
        # Code executed inside the inner loop scope
        statements
```

Practical Example:

```
# Program to print a 3x3 grid of stars (*)
for row in range(3): # Outer Loop controls rows
    for col in range(3): # Inner Loop controls columns
        print("*", end=" ") # end=" " keeps printing on the same line
    print() # Moves to the next line after finishing a row
```

4. The `break` Statement

The `break` statement is a control flow tool used to terminate the loop prematurely immediately when it is encountered. It breaks out of the loop instantly, and program control passes directly to the very next line of code situated outside that loop.

- **Note:** If a `break` statement is executed inside a nested loop, it will only break out of the innermost loop it belongs to.

Syntax:

```
while condition:
    if control_condition:
        break
    statements
```

Practical Example:

```
# Program to stop searching when a specific item is found
for target in range(1, 10):
    if target == 5:
```

```

    print("Target number 5 found! Breaking out of loop early.")
    break
    print("Checking number:", target)

print("Execution outside the loop continues here.")

```

5. The `continue` Statement

The `continue` statement is used to skip the remaining statements inside the current iteration of a loop and force the program control to jump directly to the next iteration cycle.

- **Key Difference:** `break` stops the loop completely; `continue` only skips the rest of the code block for that *single turn* and moves on to the next loop check.

Syntax:

```

for variable in sequence:
    if control_condition:
        continue
    statements # These lines are skipped if continue executes

```

Practical Example:

```

# Program to print numbers from 1 to 5, skipping 3
for num in range(1, 6):
    if num == 3:
        continue # Skips printing 3 and moves straight to num = 4
    print("Number:",

```

6. The `pass` Statement

In Python, structural syntax blocks (like loop bodies, functions, or classes) cannot remain completely empty. If you create a loop statement but don't add any code inside its block, Python will throw an `IndentationError`.

The `pass` statement is a null statement. When it executes, absolutely nothing happens. It acts purely as a syntactic placeholder to write empty loop blocks or structures that you intend to develop in the future.

Syntax:

```

for variable in sequence:
    pass # Placeholder to prevent structural errors

```

Practical Example:

```

# Code skeleton intended for future execution implementation
for element in range(100):
    pass # Interpreter skips this without errors; ready to fill later

print("The empty pass loop executed smoothly without crashing.")

```

Unit 7: Strings

In Python, a String is a contiguous sequence of characters enclosed within quotation marks. Strings are one of the most widely used core data types in Python and are used to store and manipulate text-based information (such as names, addresses, or messages).

1. String Creation

Python provides great flexibility when creating strings. You can define a string literal using single quotes, double quotes, or triple quotes.

- Single Quotes ('...') & Double Quotes ("..."): These are used interchangeably for standard, single-line strings.
 - *Tip:* Double quotes are ideal if the text itself contains a single quote or an apostrophe.
- Triple Quotes ('''...''' or """..."""): These are utilized to create multi-line strings exactly as typed, preserving all line breaks and whitespaces. They are also used for internal document notes (docstrings).

Practical Example:

```
# 1. Using single and double quotes
center = 'Open Technology'
course = "Python Programming"

# Solving apostrophe issues cleanly
sentence = "Welcome to Kanika's Python lecture."

# 2. Using triple quotes for multi-line blocks
address = """Open Technology Computer Education,
Main Market,
Workstation Hub."""
```

2. String Indexing

Indexing is used to access a single, specific character from a string based on its position numerical coordinate. Python strings use a zero-indexed tracking scheme.

- Positive Indexing (Left-to-Right): Begins at 0 for the very first character and scales upwards to `Length - 1`.
- Negative Indexing (Right-to-Left): Begins at -1 for the final trailing character and moves backwards down to `-Length`. This is unique to Python and allows easy access to the end of a string.

Practical Example:

```
word = "PYTHON"

# Positive Indexing
print(word[0]) # Output: P
print(word[2]) # Output: T
```

```
# Negative Indexing
print(word[-1]) # Output: N (Last character)
print(word[-4]) # Output: T
```

Note: Attempting to access an index beyond the string length (e.g., `word[10]`) will instantly trigger an `IndexError: string index out of range`.

3. String Slicing

Slicing is used to extract a subset or a specific portion (substring) from an existing string.

Syntax:

```
string_variable[start : stop : step]
```

- **start**: The index where the slice begins (Included). Default is **0**.
- **stop**: The index where the slice ends (Excluded—extracts up to **stop - 1**). Default is the end of the string.
- **step**: The increment value determining which characters to skip. Default is **1**.

Practical Examples:

Python

```
msg = "OpenTechnology"

# 1. Basic Slice (Extract index 0 to 3)
print(msg[0:4]) # Output: Open

# 2. Omitting Start/Stop parameters
print(msg[:4]) # Output: Open (Starts from 0 automatically)
print(msg[4:]) # Output: Technology (Goes to the very end)

# 3. Utilizing Step parameter (Extract every alternate character)
print(msg[0:14:2]) # Output: OeTehoo

# 4. Reversing a String (A classic Interview/Viva trick using negative step)
reversed_msg = msg[::-1]
print(reversed_msg) # Output: ygolonhceTnepO
```

4. String Operations

Python allows you to combine and repeat strings cleanly using standard mathematical operators.

- **Concatenation (+ Operator):** Joins or merges two distinct string objects together into a single continuous string. *Note: You cannot concatenate a string with a number directly without typecasting.*
- **Repetition (* Operator):** Repeats a string sequence a specified number of times.

Practical Example:

```
str1 = "Open"
str2 = "Tech"

# Concatenation operation
full_name = str1 + " " + str2
print(full_name) # Output: Open Tech

# Repetition operation
separator = "*" * 10
print(separator) # Output: *****
```

5. String Methods

String methods are built-in functions that you can call on a string object to analyze or transform its text content.

CRITICAL ARCHITECTURAL RULE: Because Python strings are immutable (cannot be altered in place), string methods *never modify the original variable*. Instead, they always return a brand-new, transformed string object.

Here are the most widely used string methods grouped by function:

A. Case Modification Methods

- **lower():** Converts all characters to lowercase.
- **upper():** Converts all characters to uppercase.
- **title():** Capitalizes the first letter of every individual word.
- **capitalize():** Capitalizes only the very first letter of the entire string.

B. Search & Inspection Methods

- **find(substring):** Searches for a substring and returns the lowest index where it begins. Returns **-1** if it is not found.
- **count(substring):** Returns the total number of times a substring appears in the string.

- **startswith(prefix) / endswith(suffix)**: Returns a Boolean value (**True/False**) indicating whether the string begins or ends with the specified character sequence.

C. Cleaning & Splitting Methods

- **strip()**: Removes all leading and trailing whitespaces or blank tabs from the string.
- **replace(old, new)**: Finds all occurrences of the specified old substring and replaces them with the new substring.
- **split(separator)**: Breaks the string down into individual substrings based on the separator token and returns them grouped inside a List.

Method Execution Code Snippet:

```
sample_text = " python programming language "
```



```
# Cleaning and Case shifting
clean_text = sample_text.strip()
print(clean_text.upper())      # Output: PYTHON PROGRAMMING LANGUAGE
```



```
# Replacing text patterns
updated_text = clean_text.replace("language", "course")
print(updated_text.title())    # Output: Python Programming Course
```



```
# Splitting string text into a List
words_list = clean_text.split(" ")
print(words_list)             # Output: ['python', 'programming', 'language']
```



```
# Inspection queries
print(clean_text.startswith("py")) # Output: True
print(clean_text.count("g"))      # Output: 2 (Counts 'g' in programming)
```

Unit 7: Strings

In Python, a String is a contiguous sequence of characters enclosed within quotation marks. Strings are one of the most widely used core data types in Python and are used to store and manipulate text-based information (such as names, addresses, or messages).

1. String Creation

Python provides great flexibility when creating strings. You can define a string literal using single quotes, double quotes, or triple quotes.

- Single Quotes (' ... ') & Double Quotes (" ... "): These are used interchangeably for standard, single-line strings.
 - *Tip:* Double quotes are ideal if the text itself contains a single quote or an apostrophe.
- Triple Quotes (' ' ' ... ' ' ' or " " " ... " " "): These are utilized to create multi-line strings exactly as typed, preserving all line breaks and whitespaces. They are also used for internal document notes (docstrings).

Practical Example:

```
# 1. Using single and double quotes
center = 'Open Technology'
course = "Python Programming"

# Solving apostrophe issues cleanly
sentence = "Welcome to Kanika's Python lecture."

# 2. Using triple quotes for multi-line blocks
address = """Open Technology Computer Education,
Main Market,
Workstation Hub."""
```

2. String Indexing

Indexing is used to access a single, specific character from a string based on its position numerical coordinate. Python strings use a zero-indexed tracking scheme.

- Positive Indexing (Left-to-Right): Begins at **0** for the very first character and scales upwards to **Length - 1**.
- Negative Indexing (Right-to-Left): Begins at **-1** for the final trailing character and moves backwards down to **-Length**. This is unique to Python and allows easy access to the end of a string.

Practical Example:

```
word = "PYTHON"

# Positive Indexing
print(word[0]) # Output: P
print(word[2]) # Output: T

# Negative Indexing
print(word[-1]) # Output: N (Last character)
print(word[-4]) # Output: T
```

Note: Attempting to access an index beyond the string length (e.g., `word[10]`) will instantly trigger an `IndexError: string index out of range`.

3. String Slicing

Slicing is used to extract a subset or a specific portion (substring) from an existing string.

Syntax:

```
string_variable[start : stop : step]
```

- **start**: The index where the slice begins (Included). Default is **0**.
- **stop**: The index where the slice ends (Excluded—extracts up to **stop - 1**). Default is the end of the string.
- **step**: The increment value determining which characters to skip. Default is **1**.

Practical Examples:

```
msg = "OpenTechnology"

# 1. Basic Slice (Extract index 0 to 3)
print(msg[0:4]) # Output: Open

# 2. Omitting Start/Stop parameters
print(msg[:4]) # Output: Open (Starts from 0 automatically)
print(msg[4:]) # Output: Technology (Goes to the very end)

# 3. Utilizing Step parameter (Extract every alternate character)
print(msg[0:14:2]) # Output: OeTehoo

# 4. Reversing a String (A classic Interview/Viva trick using negative step)
reversed_msg = msg[::-1]
print(reversed_msg) # Output: ygolonhceTnepO
```

4. String Operations

Python allows you to combine and repeat strings cleanly using standard mathematical operators.

- **Concatenation (+ Operator)**: Joins or merges two distinct string objects together into a single continuous string. *Note: You cannot concatenate a string with a number directly without typecasting.*
- **Repetition (* Operator)**: Repeats a string sequence a specified number of times.

Practical Example:

```
str1 = "Open"
str2 = "Tech"

# Concatenation operation
full_name = str1 + " " + str2
print(full_name) # Output: Open Tech

# Repetition operation
separator = "*" * 10
print(separator) # Output: **********
```

5. String Methods

String methods are built-in functions that you can call on a string object to analyze or transform its text content.

CRITICAL ARCHITECTURAL RULE: Because Python strings are immutable (cannot be altered in place), string methods *never modify the original variable*. Instead, they always return a brand-new, transformed string object.

Here are the most widely used string methods grouped by function:

A. Case Modification Methods

- **lower()**: Converts all characters to lowercase.
- **upper()**: Converts all characters to uppercase.
- **title()**: Capitalizes the first letter of every individual word.
- **capitalize()**: Capitalizes only the very first letter of the entire string.

B. Search & Inspection Methods

- **find(substring)**: Searches for a substring and returns the lowest index where it begins. Returns **-1** if it is not found.
- **count(substring)**: Returns the total number of times a substring appears in the string.
- **startswith(prefix) / endswith(suffix)**: Returns a Boolean value (**True/False**) indicating whether the string begins or ends with the specified character sequence.

C. Cleaning & Splitting Methods

- **strip()**: Removes all leading and trailing whitespaces or blank tabs from the string.

- **replace(old, new)**: Finds all occurrences of the specified old substring and replaces them with the new substring.
- **split(separator)**: Breaks the string down into individual substrings based on the separator token and returns them grouped inside a List.

Method Execution Code Snippet:

```
sample_text = " python programming language "
```



```
# Cleaning and Case shifting
clean_text = sample_text.strip()
print(clean_text.upper())      # Output: PYTHON PROGRAMMING LANGUAGE
```



```
# Replacing text patterns
updated_text = clean_text.replace("language", "course")
print(updated_text.title())    # Output: Python Programming Course
```



```
# Splitting string text into a List
words_list = clean_text.split(" ")
print(words_list)              # Output: ['python', 'programming', 'language']
```



```
# Inspection queries
print(clean_text.startswith("py")) # Output: True
print(clean_text.count("g"))      # Output: 2 (Counts 'g' in programming)
```

Here are the complete, modern, and highly structured study notes for Unit 8: Lists in clean English. The formatting is fully optimized so you can copy and paste them directly into Microsoft Word or Excel for your students.

Unit 8: Lists

In Python, a List is a built-in data structure used to store a collection of items in a specific sequence. Lists are one of the most powerful and heavily used data types in Python because they can hold elements of different data types simultaneously and can change size dynamically.

1. Creating Lists

Lists are created by placing all the items (elements) inside square brackets [], separated by commas.

- **Key Characteristics of Lists:**
 - **Ordered Sequence:** Items in a list maintain a defined order. New items are added to the end of the list.
 - **Mutable:** Unlike strings, lists are *mutable*, meaning you can change, add, or remove elements after the list has been created without creating a brand-new object.
 - **Heterogeneous Elements:** A single list can contain mixed data types, such as integers, floats, strings, and even other lists.

Practical Example:

```
# 1. Creating an empty list
empty_list = []

# 2. Creating a list of integers
roll_numbers = [101, 102, 103, 104]

# 3. Creating a mixed (heterogeneous) data list
student_profile = [101, "Alice", 85.5, True]
# Structure: [Int, String, Float, Boolean]

# 4. Creating a nested list (a list inside a list)
matrix = [[1, 2], [3, 4]]
```

2. List Indexing

List indexing works exactly like string indexing. It allows you to access individual items within a list using their numerical position coordinates. Python lists use a zero-indexed tracking scheme.

- **Positive Indexing (Left-to-Right):** Starts at `0` for the first item and increases to `Length - 1`.
- **Negative Indexing (Right-to-Left):** Starts at `-1` for the last item and moves backward down to `-Length`.

Practical Example:

```
courses = ["C++", "Java", "Python", "PHP"]

# Accessing elements using positive indices
print(courses[0]) # Output: C++
print(courses[2]) # Output: Python

# Accessing elements using negative indices
print(courses[-1]) # Output: PHP (Last element)
print(courses[-3]) # Output: Java
```

Note: Attempting to access an index that does not exist in the list (e.g., `courses[10]`) will instantly trigger an `IndexError: list index out of range`.

3. List Slicing

Slicing allows you to extract a specific subset or a smaller sub-list from an existing list.

Syntax:

```
list_variable[start : stop : step]
```

- **start:** The index where the slice begins (Included). Default is `0`.
- **stop:** The index where the slice ends (Excluded—extracts up to `stop - 1`).
- **step:** The increment value determining which elements to skip. Default is `1`.

Practical Examples:

```
numbers = [0, 10, 20, 30, 40, 50, 60]

# 1. Extracting a sub-list (Index 1 to 4)
print(numbers[1:5]) # Output: [10, 20, 30, 40]

# 2. Omitting start/stop parameters
```

```
print(numbers[:3]) # Output: [0, 10, 20] (First 3 elements)
print(numbers[4:]) # Output: [40, 50, 60] (From index 4 to the end)

# 3. Using a custom step parameter
print(numbers[0:7:2]) # Output: [0, 20, 40, 60] (Every alternate element)

# 4. Reversing a list using a negative step
print(numbers[::-1]) # Output: [60, 50, 40, 30, 20, 10, 0]
```

4. List Operations

Python allows you to combine, repeat, and manipulate lists cleanly using standard mathematical operators and keywords.

- **Concatenation (+ Operator):** Joins or merges two distinct list objects together into a single continuous list.
- **Repetition (* Operator):** Repeats the elements of a list a specified number of times.
- **Membership Check (in & not in Operators):** Evaluates whether a specific item exists inside the list, returning **True** or **False**.

Practical Example:

```
batch1 = ["A", "B"]
batch2 = ["C", "D"]

# Concatenation
combined_batch = batch1 + batch2
print(combined_batch) # Output: ['A', 'B', 'C', 'D']

# Repetition
repeated_pattern = batch1 * 3
print(repeated_pattern) # Output: ['A', 'B', 'A', 'B', 'A', 'B']

# Membership Verification
print("A" in combined_batch) # Output: True
print("Z" not in combined_batch) # Output: True
```

5. List Methods

Because lists are mutable, Python provides several built-in methods to directly modify, organize, and inspect the items inside a list.

Unlike string methods, list methods modify the original list directly in memory and usually return **None**.

A. Adding Elements

- **append(item):** Adds an item to the very end of the list.
- **insert(index, item):** Inserts an item at a specific index position, shifting subsequent items to the right.
- **extend(iterable):** Appends all items from another collection or list to the end of the current list.

B. Removing Elements

- **pop():** Removes and returns the last item from the list. If an index is provided (e.g., **pop(0)**), it removes the item at that specific index.

- `remove(item)`: Searches for and removes the first occurrence of a specific item value. Throws a `ValueError` if the item is not found.
- `clear()`: Removes all elements from the list, leaving it completely empty.

C. Organizing & Utility Methods

- `sort()`: Sorts the items of the list in place in ascending order (can be set to descending using `sort(reverse=True)`).
- `reverse()`: Reverses the order of the elements in the list in place.
- `count(item)`: Returns the total number of times a specific item value appears in the list.
- `index(item)`: Returns the index position of the first occurrence of a specific item value.

Method Execution Code Snippet:

```
items = ["Mouse", "Keyboard", "Monitor"]

# 1. Appending and Inserting items
items.append("RAM")
print(items) # Output: ['Mouse', 'Keyboard', 'Monitor', 'RAM']

items.insert(1, "CPU")
print(items) # Output: ['Mouse', 'CPU', 'Keyboard', 'Monitor', 'RAM']

# 2. Removing items
removed_item = items.pop()
print(f"Popped: {removed_item}") # Output: Popped: RAM
print(items) # Output: ['Mouse', 'CPU', 'Keyboard', 'Monitor']

items.remove("CPU")
print(items) # Output: ['Mouse', 'Keyboard', 'Monitor']

# 3. Sorting and Utility tracking
scores = [45, 12, 89, 34, 12]
scores.sort()
print(scores) # Output: [12, 12, 34, 45, 89] (Sorted ascending)

print(scores.count(12)) # Output: 2
print(scores.index(89)) # Output: 4
```

Here are the complete, modern, and highly structured study notes for Unit 9: Tuples in clean English. The formatting is optimized so you can copy and paste them directly into Microsoft Word or Excel for your students.

Unit 9: Tuples

In Python, a Tuple is a built-in collection data type used to store a sequence of items. Tuples are very similar to lists, but they have one critical difference that changes how they are used in memory and software architecture: Tuples are immutable.

1. Creating Tuples

Tuples are created by placing all the items (elements) inside parentheses `()`, separated by commas.

- **Key Characteristics of Tuples:**
 - **Ordered Sequence:** Items in a tuple maintain a strict, defined order that cannot be shuffled automatically.
 - **Immutable:** Once a tuple is created, its elements cannot be changed, added, replaced, or deleted. This provides data security for values that must remain constant throughout the program.
 - **Heterogeneous Elements:** Like lists, a single tuple can store multiple mixed data types (integers, strings, floats, booleans) simultaneously.
 - **Allows Duplicates:** Tuples can contain identical duplicate values.

The Single Element Tuple Rule (Crucial Syntax Catch):

If you want to create a tuple containing only one single item, you must include a trailing comma after that item. If you omit the comma, Python will mistake the expression for a standard variable enclosed in algebraic parentheses.

Practical Example:

```
# 1. Creating an empty tuple
empty_tuple = ()

# 2. Creating a standard tuple of data
hardware_constants = ("Mouse", "Keyboard", "Gateway")

# 3. Creating a single-element tuple (Note the trailing comma)
valid_single_tuple = ("OpenTech",) # Inferred as <class 'tuple'>
invalid_tuple = ("OpenTech")      # Inferred as <class 'str'> !!!

# 4. Creating a tuple without parentheses (Tuple Packing)
packed_tuple = 10, 20, "Alice"
print(type(packed_tuple)) # Output: <class 'tuple'>
```

2. Tuple Operations

While you cannot modify the contents of a tuple, you can read, manipulate, combine, and query them using standard operators.

A. Tuple Indexing & Slicing

Tuple index tracking functions exactly like list tracking. It supports Positive Indexing (left-to-right, starting at `0`) and Negative Indexing (right-to-left, starting at `-1`). Slicing extracts a new, sub-tuple using the standard `[start:stop:step]` parameters.

```
data = (10, 20, 30, 40, 50)

print(data[0]) # Indexing Output: 10
print(data[-1]) # Indexing Output: 50
print(data[1:4]) # Slicing Output: (20, 30, 40)
```

B. Concatenation & Repetition

- **Concatenation (+ Operator):** Combines two or more distinct tuples together to output a *brand-new composite tuple*.
- **Repetition (* Operator):** Duplicates the elements of a tuple a specified number of times into a new tuple.

```
t1 = (1, 2)
```

```
t2 = (3, 4)

# Concatenation
combined = t1 + t2
print(combined) # Output: (1, 2, 3, 4)

# Repetition
repeated = t1 * 3
print(repeated) # Output: (1, 2, 1, 2, 1, 2)
```

C. Membership Verification

Using `in` and `not in`, you can quickly scan a tuple to check if a specific data element exists within it.

```
colors = ("Red", "Green", "Blue")
print("Green" in colors) # Output: True
```

3. Tuple Methods

Because tuples are structurally protected from modification, they do not possess destructive methods like `.append()`, `.extend()`, `.remove()`, or `.pop()`.

Python provides only two built-in methods for tuple objects, which are used purely for data inspection and tracking:

A. The `count()` Method

Returns the total number of times a specified element appears inside the tuple.

- Syntax: `tuple.count(element)`

B. The `index()` Method

Searches the tuple for a specified element and returns the first index position where it is found. If the element is missing, it raises a `ValueError`.

- Syntax: `tuple.index(element)`

Practical Method Execution:

```
sample_tuple = (10, 20, 30, 20, 40, 20)

# 1. Counting occurrences
count_of_20 = sample_tuple.count(20)
print(f"20 appears {count_of_20} times.") # Output: 20 appears 3 times.

# 2. Finding index tracking points
first_position_of_20 = sample_tuple.index(20)
print(f"First index of 20 is: {first_position_of_20}") # Output: First index of 20 is: 1

position_of_40 = sample_tuple.index(40)
print(f"Index of 40 is: {position_of_40}") # Output: Index of 40 is: 4
```

Summary: When to Use a Tuple vs. a List

To help students understand the architectural decision behind choosing a Tuple over a List, use this quick-reference comparison matrix:

Feature	List ([])	Tuple (())

Mutability	Mutable (Can change contents)	Immutable (Cannot change contents)
Performance	Slower (Requires extra memory allocation)	Faster (Highly optimized, memory efficient)
Methods	Many methods (append, pop, sort, etc.)	Only 2 inspection methods (count, index)
Use Case	For dynamic data that changes frequently.	For static read-only configurations (e.g., coordinates, database credentials).

Unit 10: Sets

In Python, a Set is a built-in collection data type used to store multiple items in a single variable. While lists and tuples are ordered sequences, a set is fundamentally different because it models a mathematical set structure.

A set is an unordered, unindexed, and mutable collection that does not allow duplicate values.

1. Creating Sets

Sets are created by placing all the items (elements) inside curly braces {}, separated by commas, or by using the built-in `set()` constructor function.

- **Key Characteristics of Sets:**
 - **Unordered:** The items in a set do not have a defined tracking order. Every time you print or use a set, the elements may appear in a completely different order. Because of this, sets do not support indexing or slicing.

- **No Duplicates Allowed:** If you try to add the same item twice to a set, Python will automatically discard the duplicate. Every element in a set must be unique.
- **Unchangeable (but Mutable):** Set elements themselves must be of an immutable data type (like integers, floats, strings, or tuples). You cannot put a mutable list inside a set. However, the set object itself is mutable—meaning you can add or remove items from it freely.

The Empty Set Trap (Crucial Syntax Catch):

In Python, curly braces `{}` are also used to create dictionaries. If you write `empty_collection = {}`, Python will infer its type as a Dictionary (`dict`), not a set. To create a completely empty set, you must use the `set()` constructor function.

Practical Example:

```
# 1. Creating a standard set of data
course_codes = {"PY01", "JV02", "C03", "PY01"}
print(course_codes) # Output: {'C03', 'JV02', 'PY01'} (Duplicates are automatically removed!)

# 2. Creating an empty set vs. an empty dictionary
invalid_empty_set = {} # This creates a DICTIONARY (<class 'dict'>)
valid_empty_set = set() # This creates a SET (<class 'set'>)

# 3. Converting a List into a Set (Excellent way to remove duplicate items from a list)
numbers_list = [1, 2, 2, 3, 4, 4, 4, 5]
unique_numbers_set = set(numbers_list)
print(unique_numbers_set) # Output: {1, 2, 3, 4, 5}
```

2. Set Operations (Mathematical Concepts)

One of the greatest advantages of using sets in Python is that you can perform standard mathematical set operations like Union, Intersection, Difference, and Symmetric Difference instantly using built-in operators or methods.

For the examples below, let's assume we have two sets representing software skills requested by different computer batches:

```
set_A = {1, 2, 3, 4}
set_B = {3, 4, 5, 6}
```

A. Union

The Union of two sets combines all unique elements from both collections.

- Operator: `|`
- Method: `set_A.union(set_B)`

```
print(set_A | set_B) # Output: {1, 2, 3, 4, 5, 6}
```

B. Intersection

The Intersection extracts only the common elements that exist in both sets simultaneously.

- Operator: `&`
- Method: `set_A.intersection(set_B)`

```
print(set_A & set_B) # Output: {3, 4}
```

C. Difference

The Difference returns the elements that exist in `set_A` but are not present in `set_B`.

- Operator: `-`

- Method: `set_A.difference(set_B)`

```
print(set_A - set_B) # Output: {1, 2}
```

D. Symmetric Difference

The Symmetric Difference returns all elements that are unique to each set—meaning elements that are in either of the sets, but not in both. It excludes the intersection items.

- Operator: `^`
- Method: `set_A.symmetric_difference(set_B)`

```
print(set_A ^ set_B) # Output: {1, 2, 5, 6}
```

3. Set Methods

Python provides many built-in utility methods to modify, alter, and inspect sets during program runtime.

A. Adding Items

- `add(item)`: Adds a single, unique element to the set. If the item already exists, the set remains unchanged.
- `update(iterable)`: Adds multiple elements (from a list, tuple, or another set) to the current set.

```
tech_stack = {"Python", "Django"}
tech_stack.add("Flask") # Adding a single item
tech_stack.update(["SQL", "Git"]) # Adding multiple items at once
print(tech_stack) # Output: {'Python', 'Django', 'Flask', 'SQL', 'Git'} (Order varies)
```

B. Removing Items

- `remove(item)`: Removes the specified item from the set. Crucial: If the item does not exist in the set, it throws a `KeyError`.
- `discard(item)`: Removes the specified item from the set. Crucial: Unlike `remove`, if the item is missing, it skips it silently and does not throw any error. This makes it safer for dynamic user inputs.
- `pop()`: Removes and returns an arbitrary (random) element from the set because sets are unordered. If the set is empty, it raises a `KeyError`.
- `clear()`: Wipes out the entire set, leaving it completely empty.

```
students = {"Alice", "Bob", "Charlie"}

students.discard("Zeta") # No error is raised even though "Zeta" isn't there
students.remove("Alice") # "Alice" is removed successfully

popped_student = students.pop() # Removes a random student entry
print(f"Removed: {popped_student}")
```

C. Evaluation & Checking Methods

- `issubset()`: Returns `True` if all elements of the current set exist within another target set.
- `issuperset()`: Returns `True` if the current set contains all elements of a target set.
- `isdisjoint()`: Returns `True` if two sets have completely zero common items in their intersection block.

```
small_set = {1, 2}
large_set = {1, 2, 3, 4}
```

```
print(small_set.issubset(large_set)) # Output: True
print(large_set.issuperset(small_set)) # Output: True
print(small_set.isdisjoint({7, 8})) # Output: True (No overlapping items)
```

Unit 11: Dictionaries

In Python, a Dictionary is a powerful built-in data structure used to store data collections in key-value pairs. Unlike lists, tuples, or sets, where elements are accessed using numerical index positions, a dictionary maps unique keys directly to their corresponding values.

A dictionary is ordered, mutable, and does not allow duplicate keys.

1. Creating Dictionaries

Dictionaries are created by placing key-value pairs inside curly braces {}, where each key is separated from its value by a colon (:), and individual pairs are separated by commas.

- **Key Characteristics of Dictionaries:**
 - **Ordered:** Starting from Python 3.7, dictionaries maintain the exact insertion order of items. New elements are appended to the end of the collection.
 - **Mutable:** Dictionaries are *mutable*, meaning you can add, modify, or remove key-value records dynamically without creating a brand-new dictionary object.
 - **Unique Keys:** Keys within a dictionary must be entirely unique. If you attempt to assign a value to an existing key, the new value will overwrite the old one.
 - **Key vs. Value Restrictions:** Keys must belong to an *immutable* data type (such as strings, integers, or tuples). Values, however, can be of any data type, including lists, sets, or even other dictionaries.

Practical Example:

```
# 1. Creating an empty dictionary
empty_dict = {}

# 2. Creating a standard dictionary mapping course information
course_details = {
    "course_id": "PY101",
    "title": "Python Programming",
    "duration_months": 3,
    "fee": 4500
}

# 3. Demonstrating unique keys (the second entry will overwrite the first)
student_scores = {
    "Alice": 85,
    "Bob": 92,
    "Alice": 95 # Overwrites Alice's score of 85
}

print(student_scores) # Output: {'Alice': 95, 'Bob': 92}
```

2. Dictionary Operations

Python provides highly intuitive syntax mechanisms to look up, access, modify, and evaluate items within a dictionary.

A. Accessing and Modifying Values

You can access a value by passing its corresponding key inside square brackets `[]`. Similarly, you can modify an existing value or create a brand-new pair using the same assignment operator syntax.

```
instructor_profile = {"name": "Kanika Thakur", "center": "Open Technology"}

# Accessing a value
print(instructor_profile["name"]) # Output: Kanika Thakur

# Modifying a value
instructor_profile["center"] = "Open Tech Workstation Hub"

# Adding a new key-value pair
instructor_profile["subject"] = "Python Core"
print(instructor_profile)
# Output: {'name': 'Kanika Thakur', 'center': 'Open Tech Workstation Hub', 'subject': 'Python Core'}
```

Note: If you attempt to access a value using a key that does not exist via the square bracket method (e.g., `instructor_profile["age"]`), Python will halt program execution and raise a `KeyError`.

B. Membership Testing (`in` & `not in`)

Membership operators test for the presence of Keys within a dictionary, not values.

```
inventory = {"Mouse": 15, "Keyboard": 10}

print("Mouse" in inventory) # Output: True
print(15 in inventory)     # Output: False (15 is a value, not a key)
```

3. Dictionary Methods

Python provides an extensive set of built-in methods to view, alter, extract, and secure data within a dictionary structure.

A. Extraction Methods (Views)

- `keys()`: Returns a dynamic view object containing all the unique keys in the dictionary.
- `values()`: Returns a dynamic view object containing all the values in the dictionary.
- `items()`: Returns a view object containing each key-value pair as a independent Tuple element grouped inside a list-like collection.

```
batch = {"Stud1": "Alice", "Stud2": "Bob"}

print(batch.keys()) # Output: dict_keys(['Stud1', 'Stud2'])
print(batch.values()) # Output: dict_values(['Alice', 'Bob'])
print(batch.items()) # Output: dict_items([('Stud1', 'Alice'), ('Stud2', 'Bob')])
```

B. Safe Retrieval & Alteration Methods

- `get(key, default)`: Safely extracts the value associated with the specified key. If the key is missing from the dictionary, instead of crashing with a `KeyError`, it returns `None` (or a custom default fallback value that you specify).
- `update(iterable)`: Merges the current dictionary with another dictionary or an iterable of key-value pairs, overwriting values for matching keys.

```
config = {"theme": "Dark", "font": "Consolas"}

# Safe retrieval without KeyError
print(config.get("theme"))      # Output: Dark
print(config.get("resolution", "1080p")) # Output: 1080p (Fallback value)

# Merging dictionaries
config.update({"font": "Arial", "language": "EN"})
print(config) # Output: {'theme': 'Dark', 'font': 'Arial', 'language': 'EN'}
```

C. Removal Methods

- `pop(key)`: Removes the specified key and returns its corresponding value. Raises a `KeyError` if the key is missing unless a fallback argument is defined.
- `popitem()`: Removes and returns the last inserted key-value pair as a tuple.
- `clear()`: Empties the entire dictionary, leaving it completely blank.

```
device_counts = {"Printers": 2, "Monitors": 8, "UPS": 4}

# Pop specific item
removed_monitors = device_counts.pop("Monitors")
print(f"Removed Monitors: {removed_monitors}") # Output: Removed Monitors: 8

# Pop last item
last_item = device_counts.popitem()
print(last_item) # Output: ('UPS', 4)

print(device_counts) # Output: {'Printers': 2}
```

Summary: Core Python Collections Comparison

To assist students in selecting the correct structural mapping type for their data operations, reference this master evaluation matrix:

Collection	Brackets	Ordered	Mutable	Duplicates	Access Method
List	<code>[]</code>	Yes	Yes	Yes	Index Slicing (e.g., <code>[0]</code>)

Tuple	<code>()</code>	Yes	No	Yes	Index Slicing (e.g., <code>[0]</code>)
Set	<code>{}</code>	No	Yes	No	Cannot look up individually
Dictionary	<code>{}</code>	Yes	Yes	No (Keys)	Key Mapping (e.g., <code>["key"]</code>)

Unit 12: Functions

In programming, a Function is a self-contained, reusable block of organized code designed to perform a single, specific task. Instead of writing the same logic multiple times throughout a program, you can wrap the code inside a function and execute it whenever needed.

- **Advantages of Using Functions:**
 - **Code Reusability:** Write code once and use it many times.
 - **Reduces Code Redundancy:** Eliminates repetitive lines of code, saving memory and effort.
 - **Modularity:** Breaks large, complex programs into smaller, manageable, and organized modules.
 - **Easier Debugging:** If an error occurs, you only need to fix the code inside the specific function rather than updating multiple places.

1. Function Definition

To define a function in Python, we use the `def` keyword, followed by a unique function name, parentheses `()`, and a colon `:`. Any statements inside the function must be properly indented.

- **Syntax:**

```
def function_name(parameters):
```

```
"""Optional Docstring (documentation notes)"""
```

```
# Body of the function
```

```
statements
```

- **def:** The structural keyword used to initialize a function definition.
- **parameters:** Optional variables inside the parentheses that act as placeholders for data that the function accepts as input.

2. Function Calling

Defining a function simply creates the block of code; it does not execute it. To execute the code inside a function, you must "call" or "invoke" it by writing the function's name followed by parentheses.

Practical Example:

```
# 1. Defining the function
```

```
def greet_students():
```

```
    print("Welcome to Open Technology Computer Education!")
```

```
    print("Let's learn Python Functions today.")
```

```
# 2. Calling the function (This triggers the code inside)
```

```
greet_students()
```

```
# Calling it a second time to show reusability
```

```
greet_students()
```

3. Function Arguments (Parameters vs. Arguments)

To pass data into a function, we use parameters and arguments. While often used interchangeably, there is a distinct structural difference between them:

- **Parameters:** The placeholder variables listed inside the parentheses in the function definition.
- **Arguments:** The actual values sent to the function when it is called.

```
# 'name' is the PARAMETER
```

```
def welcome_user(name):
```

```
    print(f"Hello {name}, welcome to the session.")
```

```
# "Alice" and "Bob" are the ARGUMENTS
```

```
welcome_user("Alice")
```

```
welcome_user("Bob")
```

Types of Arguments in Python:

Python offers highly flexible methods to handle arguments during runtime:

- **A. Positional Arguments:** Arguments that must be passed in the exact sequential order in which their corresponding parameters were defined.
- **B. Keyword Arguments:** Arguments passed by explicitly naming the parameter prefix. This allows you to pass arguments in any order.
- **C. Default Arguments:** Parameters that take a predefined "fallback" value if no argument is supplied during the function call. # Parameter 'city' has a Default Value of "Delhi"

```
def show_profile(student_name, course, city="Delhi"):
```

```
    print(f"Student: {student_name} | Course: {course} | City: {city}")
```

```
# 1. Using Positional Arguments (Order matters)
```

```
show_profile("Amit", "Python")

# 2. Using Keyword Arguments (Order doesn't matter)
show_profile(course="Java", student_name="Rahul")

# 3. Overwriting the Default Argument
show_profile("Suresh", "Web Dev", "Mumbai")
```

4. The `return` Statement

The `return` statement is used to send a computed result or value back to the caller and exit the function immediately.

- **Key Rule:** A function can execute multiple statements inside its block, but as soon as a `return` statement is hit, the function terminates. If a function does not contain an explicit `return` statement, it automatically returns `None`.

Practical Example:

```
# Function that calculates and returns a value
def calculate_net_fee(gross_fee, discount):
    net_amount = gross_fee - discount
    return net_amount # Sends the result back

# Capturing the returned value inside a variable
payable_fee = calculate_net_fee(4500, 500)
print(f"Total Course Fee Payable: INR {payable_fee}")
```

5. Recursive Functions (Recursion)

A Recursive Function is a function that calls itself inside its own body. Recursion is a powerful programming technique used to break complex mathematical or algorithmic problems down into smaller, repetitive steps.

- **Essential Requirements for Recursion:**
 1. **Base Case (Stopping Condition):** A condition that stops the function from calling itself further. Without a base case, the function will call itself infinitely, causing a `RecursionError (Stack Overflow)`.
 2. **Recursive Case:** The part of the function where it calls itself with a modified, smaller argument, moving closer to the base case.

Practical Example (Factorial of a Number):

Mathematically, $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$.

```
def find_factorial(n):
    # 1. Base Case: If n is 1 or 0, return 1
    if n <= 1:
        return 1
    # 2. Recursive Case: n * factorial of (n-1)
    else:
        return n * find_factorial(n - 1)

# Execution evaluation
result = find_factorial(5)
print(f"The factorial of 5 is: {result}") # Output: 120
```

6. Lambda Functions (Anonymous Functions)

In Python, a Lambda Function is a small, anonymous function that is defined without a name using the `lambda` keyword instead of the standard `def` keyword.

- Key Characteristics of Lambda Functions:
 - They can take any number of arguments, but they can only contain a single expression.
 - They evaluate and return the result automatically without requiring an explicit `return` keyword.
 - They are primarily designed for quick, one-time use or to be passed as arguments inside higher-order functions like `map()`, `filter()`, or `sorted()`.

Syntax:

```
lambda arguments : expression
```

Practical Example:

```
# 1. Standard function version
def make_square(x):
    return x * x

# 2. Equivalent Lambda function version
lambda_square = lambda x : x * x

# Calling both functions yields identical results
print("Standard Function Output:", make_square(6)) # Output: 36
print("Lambda Function Output:", lambda_square(6)) # Output: 36

# 3. Lambda function with multiple arguments
calculate_sum = lambda a, b, c : a + b + c
print("Sum Total via Lambda:", calculate_sum(10, 20, 30)) # Output: 60
```

Here are the complete, modern, and highly structured study notes for Unit 13: Modules and Packages in clean English. The formatting is fully optimized so you can copy and paste them directly into Microsoft Word or Excel for your students.

Unit 13: Modules and Packages

As programs grow larger and more complex, keeping all the code in a single file becomes difficult to manage, debug, and maintain. Python solves this problem through Modules and Packages, which allow you to break down large programs into smaller, organized, and reusable files and directories.

1. User-defined Modules

A Module is simply a file containing Python definitions, functions, classes, and variables. Any standard Python file ending with a `.py` extension can be treated as a module.

A User-defined Module is a module created by the programmer to separate specific logic (e.g., database connections, mathematical calculations) into its own file.

How to Create and Use a User-defined Module:

Step 1: Create the Module File (`mymodule.py`) Save a file named `mymodule.py` and write some reusable code inside it:

```
# Content of mymodule.py
def welcome_msg(name):
    return f"Welcome {name} to Open Technology Computer Education!"

center_code = "OTCE-2026"
```

Step 2: Access the Module in your Main Program (`main.py`) In the same folder, create your main execution file and fetch the module's contents:

```
# Content of main.py
import mymodule # Ingesting the user-defined module

# Calling the function and variable from mymodule
message = mymodule.welcome_msg("Rohan")
print(message)
# Output: Welcome Rohan to Open Technology Computer Education!

print("Center Tracking ID:", mymodule.center_code)
# Output: Center Tracking ID: OTCE-2026
```

2. The `import` Statement

The `import` statement is the keyword used to gain access to the code inside a module from another file. Python provides multiple syntax variations to import modules based on your structural needs:

A. Standard `import`

Imports the entire module. To access its contents, you must use the dot (`.`) operator prefix.

```
import math
print(math.sqrt(16)) # Output: 4.0
```

B. `from ... import ...` (Specific Imports)

Imports only a specific function, variable, or class directly into your file's current workspace. This removes the need to use the module name as a prefix.

```
from math import sqrt, pi
print(sqrt(25)) # Output: 5.0 (No "math." prefix needed)
print(pi)      # Output: 3.141592653589793
```

C. `import ... as ...` (Aliasing/Renaming)

Gives a module a shorter, custom nickname (alias). This is highly useful for long module names or quick coding.

```
import mymodule as mm
print(mm.center_code) # Uses the short nickname
```

3. Built-in Modules

Python comes with a large set of pre-installed, ready-to-use modules known as the Python Standard Library. You do not need to install them via the internet; you can import them directly.

Here are four of the most heavily used built-in modules in Python:

A. The `math` Module

Provides access to mathematical functions and constants defined by the C standard.

```
import math
print(math.pow(2, 3)) # 2 raised to power 3 -> Output: 8.0
print(math.ceil(4.2)) # Rounds up -> Output: 5
print(math.floor(4.8)) # Rounds down -> Output: 4
```

B. The `random` Module

Used for generating pseudo-random numbers, picking random elements from sequences, or shuffling data.

```
import random
print(random.randint(1, 10)) # Generates a random integer between 1 and 10
print(random.random())      # Generates a random float between 0.0 and 1.0

items = ["Mouse", "Keyboard", "Monitor"]
print(random.choice(items)) # Picks a random item from the list
```

C. The `datetime` Module

Provides classes for manipulating and formatting dates and times.

```
import datetime
current_time = datetime.datetime.now()
print("Current Timestamp:", current_time)

current_date = datetime.date.today()
print("Today's Date:", current_date)
```

D. The `os` Module

Provides functions for interacting directly with the computer's Operating System (creating folders, listing files, checking paths).

```
import os
print("Current Working Directory:", os.getcwd()) # Gets present folder path
# os.mkdir("New_Lab_Exercise") # Code statement to create a new folder
```

4. Packages in Python

While a Module is a single Python file, a Package is a collection or directory of multiple related modules grouped together under a common workspace. Packages add an extra layer of organization to software engineering.

Structural Rules for a Python Package:

For a standard folder to be recognized by Python as a valid Package, it must meet specific directory conditions:

1. It must be a folder containing one or more `.py` module files.
2. The `__init__.py` file requirement: The directory *historically* must contain a file explicitly named `__init__.py`. This file can be completely empty, but its presence tells the Python interpreter that the directory is a package rather than a regular folder.

Architectural Directory Layout Example:

Imagine you are building a student management system package named `OpenTech`:

Plaintext

```
OpenTech/          # Top-level Package folder
|
├── __init__.py    # Initializes the package environment
├── admissions.py  # Module tracking new student entries
└── fee_manager.py # Module tracking billing records
```

How to Import Code from a Package:

To access functions located inside modules that sit inside a package, use dot notation tracking paths:

Python

```
# Importing a specific module from the package folder
from OpenTech import admissions
admissions.register_new_student("Rahul Sharma")

# Alternative: Direct function extraction syntax
from OpenTech.fee_manager import print_receipt
print_receipt(enrollment_no=1024)
```

Unit 14: Exception Handling

Even if a program is syntactically correct, it can still run into issues during execution. Exception Handling is a powerful programming mechanism used to anticipate, catch, and handle runtime errors gracefully, preventing an application from crashing unexpectedly.

1. Errors and Exceptions

In Python, disruptions in a program's execution flow are broadly divided into two categories: Syntax Errors and Exceptions.

A. Syntax Errors

- These occur when the programmer violates the formal grammar rules of the Python language.
- They are caught by the parser before the code runs (at compile time). A program with a syntax error cannot execute at all.
- *Example:* Forgetting a colon `:` at the end of an `if` statement or a loop.

B. Exceptions

- An exception is an error that occurs during the execution of a program (at runtime), even though the syntax is completely correct.
- When an exception occurs, Python halts program execution and generates an internal error tracking block known as an Exception Object. If left unhandled, the program crashes instantly.

Common Built-in Exceptions:

- `ZeroDivisionError`: Raised when a number is divided by zero.
- `ValueError`: Raised when a function receives an argument of the correct type but an inappropriate value (e.g., trying to pass `"abc"` into `int()`).
- `TypeError`: Raised when an operation is applied to an object of an inappropriate data type (e.g., adding a string to an integer).
- `FileNotFoundError`: Raised when attempting to open a file that does not exist in the specified directory path.

2. The `try` and `except` Blocks

The primary defensive strategy against runtime exceptions involves wrapping hazardous code inside a `try` block and defining a recovery routine inside an `except` block.

- **The `try` Block:** You place any code statements that might potentially trigger an error inside this block. Python monitors this block closely during runtime.
- **The `except` Block:** If an exception occurs inside the `try` block, Python immediately stops executing the remaining lines of the `try` block and jumps straight into the matching `except` block. This block contains your error recovery code (e.g., displaying a clean user-friendly warning message).

Syntax:

```
try:
    # Hazardous code that might cause an exception
    statements
except ExceptionName:
    # Code to handle the exception gracefully
    fallback_statements
```

Practical Example (Catching Specific Errors):

```
try:
    numerator = int(input("Enter numerator: "))
    denominator = int(input("Enter denominator: "))
    result = numerator / denominator
    print("Result of calculation:", result)

# Catching a ValueError if the user types alphabetic characters
except ValueError:
    print("Error: Invalid Input! Please enter numeric integers only.")

# Catching a ZeroDivisionError if denominator is 0
except ZeroDivisionError:
    print("Error: Mathematical Division by Zero is strictly prohibited.")
```

3. The `else` Block

The `else` block is an optional block used in tandem with `try-except`. It contains code that should execute only if no exceptions were raised inside the `try` block.

- **Why use it?** It separates your standard "happy path" code from the error-handling mechanisms. It guarantees that the code inside it will only run if the `try` block ran perfectly without any errors.

Syntax:

```
try:
    # Monitor this code
    statements
except:
    # Runs if an error happens
    error_recovery
else:
    # Runs ONLY if NO error happens
    success_path_statements
```

Practical Example:

```
try:
    file_score = int(input("Enter score out of 100: "))
except ValueError:
    print("Error: Could not convert data to a valid number.")
else:
    # This executes only if input conversion succeeded
    percentage = (file_score / 100) * 100
    print(f"Validation Successful! Calculated percentage: {percentage}%")
```

4. The `finally` Block

The `finally` block is a specialized block that always executes, regardless of whether an exception occurred, was caught, or was completely unhandled. It is guaranteed to run under all conditions.

- **Primary Purpose:** It is used to perform clean-up operations, such as closing open text files, releasing hardware resources, or disconnecting database connections, ensuring that resources are never left trapped in memory if a crash occurs.

Syntax:

```
try:
    statements
except:
    error_statements
finally:
    # This code executes NO MATTER WHAT
    cleanup_statements
```

Practical Example:

```
try:
    print("Opening system file stream...")
    # Simulating a file manipulation error
    result = 10 / 0
except ZeroDivisionError:
    print("Handling arithmetic errors internally.")
finally:
    # This block executes even though an exception occurred
    print("Closing system file stream safely. Resource memory freed.")
```

5. User-defined Exceptions (Custom Exceptions)

While Python's built-in exceptions cover standard mathematical and system failures, real-world business software often requires custom errors to enforce business logic (e.g., throwing an error if a customer inputs an age below 18 for a driving application).

- **How to create them:** In Python, you can create a custom exception by creating a new class that inherits from the built-in `Exception` class.
- **The `raise` keyword:** To trigger an exception manually based on your own custom condition, you use the `raise` keyword.

Practical Example:

```
# 1. Defining the custom exception class by inheriting from Exception
class InvalidAgeError(Exception):
    """Exception raised for errors in the input age restriction profile."""
    pass
```

```
# 2. Implementing business logic utilizing the custom exception
def verify_voter_eligibility(age):
    if age < 18:
        # Manually triggering the exception object
        raise InvalidAgeError("Age is below 18. Student is ineligible to vote.")
    else:
        print("Registration complete. Access allowed.")

# 3. Testing the user-defined exception handling architecture
try:
    student_age = int(input("Enter applicant age for computer club voting: "))
    verify_voter_eligibility(student_age)
except InvalidAgeError as error:
    print(f"Custom Application Rejection -> {error}")
except ValueError:
    print("System Message: Please enter a valid numerical age representation.")
```